

INFORME DE PROYECTO FINAL DEL CURSO DE SOLUCIONES WEB Y APLICACIONES DISTRIBUIDAS

1. PORTADA

Campo	Detalle
Título del informe	Bambeli — Desarrollo y despliegue de una aplicación web e-commerce de ropa infantil en un ambiente distribuido
Asignatura	Soluciones Web y Aplicaciones Distribuidas
Docente	_(completar)_
Fecha	_(completar)_

Integrantes del equipo

Nº	Nombres y apellidos	Rol en el equipo
1	Daniel Francisco Aguirre Espinoza	_(completar: ej. Líder / Backend)_
2	_(completar)_	_(completar)_
3	_(completar)_	_(completar)_
4	_(completar)_	_(completar)_

2. RESUMEN EJECUTIVO

Breve descripción de la aplicación desarrollada

Bambeli es una aplicación web de comercio electrónico (e-commerce) orientada a la venta de ropa infantil. Permite a los clientes navegar y filtrar un catálogo de productos, gestionar un carrito de compras y generar pedidos; mientras que el personal administrador gestiona productos, categorías, banners y pedidos desde un panel de administración. La solución se construyó como una **aplicación distribuida** con un frontend en **Angular**, un backend en **Spring Boot** (API REST) y una base de datos **SQLite**, comunicados mediante servicios web REST.

Objetivos alcanzados

- API REST funcional con arquitectura por capas (controlador → servicio → repositorio → entidad).
- CRUD completo sobre las entidades principales (productos, categorías, banners, pedidos).
- Autenticación y autorización con **Spring Security + JWT** y control de acceso por roles.
- Persistencia con **JPA / Hibernate** sobre SQLite.
- Documentación interactiva de la API con **Swagger / OpenAPI**.
- Interfaz de usuario moderna y responsiva (catálogo, carrito, login y panel admin).

Herramientas utilizadas

Categoría	Herramientas
Frontend	Angular 20, TypeScript, HTML5, CSS3 (Flexbox/Grid)
Backend	Java 21, Spring Boot 3.5, Spring MVC, Spring Data JPA, Spring Security
Seguridad	JWT (jjwt), BCrypt
Base de datos	SQLite + Hibernate
Documentación API	Springdoc OpenAPI (Swagger UI)
Build / dependencias	Maven (wrapper), npm
Control de versiones	Git + GitHub

Principales resultados

Se obtuvo una aplicación e-commerce **operativa de extremo a extremo**, con flujo de compra completo, panel administrativo con CRUD funcional, seguridad basada en JWT y documentación de API. El diseño desacoplado permite el **despliegue independiente** de cada componente y sienta las bases para una infraestructura escalable, eficiente y sostenible.

3. INTRODUCCIÓN

3.1. Justificación del proyecto

El comercio electrónico se ha convertido en un canal esencial para las pequeñas y medianas empresas. Bambeli surge de la necesidad de ofrecer a una tienda de ropa infantil una plataforma digital propia que le permita exhibir su catálogo, recibir pedidos en línea y gestionar su negocio de forma centralizada, sin depender de infraestructura física adicional ni de intermediarios.

El proyecto también responde a una necesidad académica: aplicar de forma integral los conocimientos del curso (desarrollo web frontend y backend, arquitecturas distribuidas, seguridad, persistencia y despliegue), demostrando buenas prácticas de ingeniería de software y consideraciones de sostenibilidad tecnológica alineadas con el **ODS 9 (Industria, Innovación e Infraestructura)**.

3.2. Alcance

Incluye:

- Catálogo de productos con búsqueda y filtros (categoría, talla, color, precio, stock).
- Registro, inicio de sesión y gestión de sesión con JWT.
- Carrito de compras y generación de pedidos con datos de envío.
- Panel administrativo con CRUD de productos, categorías y banners, y gestión de pedidos.
- API REST documentada y base de datos relacional.

No incluye (en esta versión):

- Pasarela de pago real.
- Despliegue productivo en la nube con dominio propio (se documenta el proceso previsto).

3.3. Objetivo general

Desarrollar y documentar una aplicación web e-commerce con el framework Spring en el backend y Angular en el frontend, cubriendo todas las fases del ciclo de vida del software, su despliegue en un ambiente distribuido y la aplicación de buenas prácticas de infraestructura sostenible y eficiente.

3.4. Objetivos específicos

1. Implementar una API REST con Spring Boot bajo arquitectura por capas (MVC).
2. Construir una interfaz de usuario moderna y responsiva con Angular.
3. Implementar el CRUD completo sobre las tablas principales del sistema.
4. Asegurar la aplicación mediante Spring Security y JWT con control de acceso por roles.
5. Persistir la información con JPA / Hibernate.
6. Documentar la API mediante Swagger / OpenAPI.
7. Documentar el despliegue en un ambiente distribuido y las consideraciones de sostenibilidad

tecnológica (ODS 9).

4. DISEÑO Y DESARROLLO DE LA APLICACIÓN

4.1. Descripción funcional de la aplicación

Módulo cliente (público / usuario registrado):

- Navegación y búsqueda en el catálogo de productos.
- Filtros por categoría, talla, color, precio y disponibilidad.
- Registro e inicio de sesión.
- Carrito de compras.
- Generación de pedidos con datos de envío (departamento, provincia, distrito, dirección).
- Consulta de sus propios pedidos.

Módulo administrador:

- Panel de control (dashboard) con métricas.
- Gestión de productos (crear, listar, editar, eliminar).
- Gestión de categorías (CRUD).
- Gestión de banners promocionales (CRUD).
- Gestión y cambio de estado de los pedidos.

4.2. Arquitectura propuesta

La aplicación adopta una **arquitectura cliente-servidor / N-capas distribuida**. No se trata de un monolito clásico: el frontend y el backend son **aplicaciones independientes y desacopladas** que se comunican exclusivamente mediante **servicios web REST (JSON)**, lo que permite desarrollarlas, escalarlas y desplegarlas por separado.

El backend, internamente, sigue el patrón **MVC en capas**: Controller → Service → Repository → Entity.

4.3. Diagrama de componentes y tecnologías usadas



Capas internas del backend:

Petición HTTP			
▼ Controller	(@RestController)	→	Recibe la petición y responde JSON
▼ Service	(@Service)	→	Lógica de negocio
▼ Repository	(JpaRepository)	→	Acceso a datos
▼ Entity	(@Entity)	→	Mapea la tabla de la base de datos

Capa	Tecnología	Versión
Frontend	Angular	20.3
Frontend	TypeScript	5.8
Backend	Java	21
Backend	Spring Boot / Spring MVC	3.5.0
Backend	Spring Data JPA / Hibernate	6.6
Backend	Spring Security + JWT (jjwt)	0.12.6
Documentación	Springdoc OpenAPI / Swagger	2.8
Base de datos	SQLite	3.46

4.4. Desarrollo backend con Spring

El backend se organiza en paquetes según su responsabilidad:

Paquete	Componentes	Función
controller	AuthController, ProductController, OrderController, CategoryController, BannerController, AdminController	Exponen los endpoints REST
service	AuthService, ProductService, OrderService	Contienen la lógica de negocio
repository	UserRepository, ProductRepository, OrderRepository, CategoryRepository, BannerRepository	Acceso a datos con JpaRepository
entity	User, Product, Category, Order, OrderItem, Banner	Entidades JPA mapeadas a tablas
dto	AuthRequest, AuthResponse, OrderRequest, etc.	Objetos de transferencia de datos

Paquete	Componentes	Función
security	JwtService, JwtAuthenticationFilter, CustomUserDetailsService	Seguridad y JWT
config	SecurityConfig, CorsConfig, DataSeeder	Configuración de la aplicación

Ejemplo — Flujo CRUD de la tabla productos:

Operación	Método HTTP	Endpoint	Método del controlador	Seguridad
CREATE	POST	/api/products	ProductController.create()	Rol ADMINISTRADOR
READ (lista)	GET	/api/products	ProductController.list()	Público
READ (uno)	GET	/api/products/{id}	ProductController.get()	Público
UPDATE	PUT	/api/products/{id}	ProductController.update()	Rol ADMINISTRADOR
DELETE	DELETE	/api/products/{id}	ProductController.delete()	Rol ADMINISTRADOR

El recorrido de una operación de escritura es: el panel Angular envía la petición con el token JWT → ProductController la recibe → ProductService aplica la lógica → ProductRepository persiste → Hibernate ejecuta el INSERT/UPDATE/DELETE sobre la tabla productos.

4.5. Uso de Spring Security y aspectos de seguridad

- **Autenticación:** el usuario inicia sesión (POST /api/auth/login) y recibe un **token JWT**.
- **Autorización por roles:** CLIENTE y ADMINISTRADOR (enum Role), aplicada con

@PreAuthorize("hasRole('ADMINISTRADOR')") en las operaciones sensibles.

- **Filtro JWT:** JwtAuthenticationFilter valida el token en cada petición protegida.
- **Cifrado de contraseñas:** BCryptPasswordEncoder.
- **Sesiones STATELESS:** el servidor no guarda estado de sesión; cada petición se autentica por token.
- **CORS:** configurado en CorsConfig para permitir la comunicación con el frontend.

Nota sobre OAuth2: en esta versión la autenticación se implementó con **JWT propio**. OAuth2 queda planteado como mejora futura (inicio de sesión con proveedores externos como Google).

Credenciales de prueba (sembradas por DataSeeder):

- Administrador: admin@bambeli.com / 12345678
- Cliente: cliente@bambeli.com / 12345678

4.6. Modelo de base de datos

Entidad (clase Java)	Tabla	Descripción
User	usuarios	Usuarios (CLIENTE / ADMINISTRADOR)
Product	productos	Productos del catálogo
Category	categorias	Categorías de productos
Order	pedidos	Pedidos de los clientes
OrderItem	_(detalle)_	Líneas de detalle de cada pedido
Banner	banners	Banners promocionales

Relaciones principales:

- Product **N:1** Category (un producto pertenece a una categoría).
- Order **N:1** User (un pedido pertenece a un usuario).
- Order **1:N** OrderItem (un pedido tiene varias líneas de detalle).

usuarios 1 ■■■■ N pedidos 1 ■■■■ N detalle_pedido N ■■■■ 1 productos N ■■■■ 1 categorias

La base de datos bambeli.db se crea automáticamente al iniciar el backend (Hibernate, ddl-auto=update) y se llena con datos demo mediante DataSeeder.

4.7. Interfaz de usuario

La interfaz se desarrolló con Angular aplicando principios de **UI/UX** y diseño **responsivo** (media queries, Flexbox y CSS Grid). Pantallas principales:

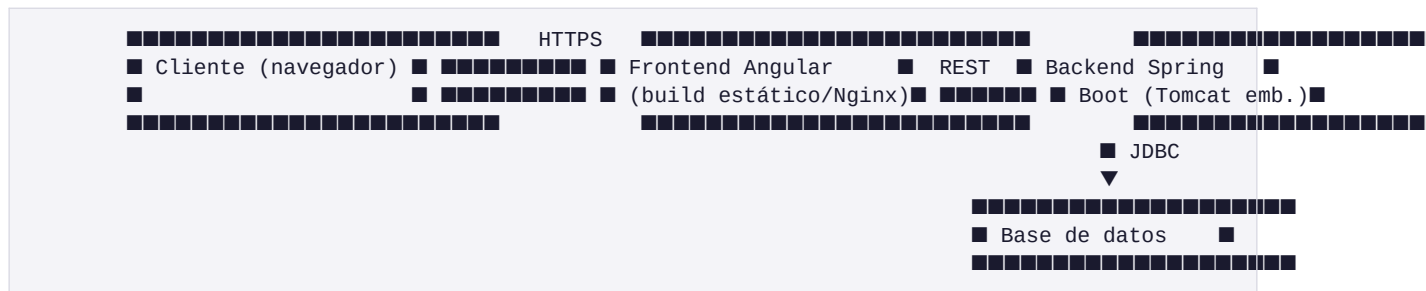
- **Inicio:** carrusel de banners y productos destacados.
- **Catálogo:** grilla de productos con buscador y filtros laterales.
- **Detalle de producto:** imagen, tallas, colores y selección de cantidad.
- **Carrito y checkout:** resumen del pedido y formulario de envío.
- **Login / Registro:** autenticación de usuarios.
- **Panel admin:** tablas y formularios para el CRUD de productos, banners y gestión de pedidos.

(Anexar capturas de pantalla de cada pantalla en la sección de anexos / bibliografía.)

5. AMBIENTES DISTRIBUIDOS Y DESPLIEGUE

5.1. Descripción del ambiente distribuido

Bambeli está concebido como una **aplicación distribuida**: sus componentes (frontend, backend y base de datos) son independientes y pueden ejecutarse en servidores o servicios distintos, comunicándose por red. Esto permite escalar y mantener cada parte por separado.



Estrategia de despliegue propuesta:

- **Frontend:** se compila a archivos estáticos (`npm run build`) y se sirve mediante **Nginx** o

un servicio de hosting/CDN.

- **Backend:** se empaqueta como un `.jar` autocontenido con **Tomcat embebido** y se ejecuta como servicio. Alternativamente puede desplegarse en **Apache Tomcat**, **WildFly** o **WebLogic**.

- **Contenerización (opcional):** cada componente puede empaquetarse en una imagen **Docker** y orquestarse con **Docker Compose** o **Kubernetes**.

- **Cloud:** los contenedores/servicios pueden alojarse en proveedores como **AWS**, **GCP**, **Azure**,

Render o **Railway**.

5.2. Configuración de servicios, contenedores y orquestación

1. Compilación del backend

```
cd backend
./mvnw clean package
java -jar target/backend-0.0.1-SNAPSHOT.jar
```

2. Compilación del frontend

```
cd frontend
npm install
npm run build # genera dist/ con archivos estáticos optimizados
```

3. Contenerización con Docker (ejemplo propuesto)

```
# Backend - Dockerfile
FROM eclipse-temurin:21-jre
COPY target/backend-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4. Orquestación con Docker Compose (ejemplo propuesto)

```
services:
  backend:
    build: ./backend
    ports: ["8080:8080"]
  frontend:
    build: ./frontend
    ports: ["80:80"]
    depends_on: [backend]
```

5. Seguridad en la comunicación (SSL/TLS) Para producción se recomienda habilitar **HTTPS** con un certificado **SSL/TLS** (por ejemplo, **Let's Encrypt** detrás de un *reverse proxy* Nginx), garantizando el cifrado de datos entre el cliente y el servidor.

6. Configuración para producción

- Externalizar credenciales y el **secret** JWT mediante **variables de entorno**.
- Restringir el **CORS** únicamente al dominio del frontend.
- Para mayor escalabilidad, migrar de SQLite a un motor gestionado (**PostgreSQL / MySQL**).

6. CONSIDERACIONES DE INFRAESTRUCTURA TECNOLÓGICA SOSTENIBLE

Esta sección documenta las buenas prácticas aplicadas y previstas para lograr una infraestructura **sostenible y eficiente**, en línea con el **ODS 9 (Industria, Innovación e Infraestructura)**.

6.1. Estrategias para optimizar el consumo de recursos

- **Arquitectura ligera:** el uso de **SQLite** en desarrollo/demostración evita un servidor de base de datos adicional, reduciendo memoria, energía y hardware.
- **Backend autocontenido:** el `.jar` con Tomcat embebido elimina servidores pesados y reduce el consumo de cómputo.
- **Contenedores eficientes:** imágenes Docker ligeras (JRE en lugar de JDK completo) y un único contenedor por servicio reducen el número de instancias.
- **Escalamiento bajo demanda:** la arquitectura desacoplada permite escalar ****solo el componente que lo requiera****, evitando el sobredimensionamiento de la infraestructura.

6.2. Elección de herramientas y servicios basados en eficiencia energética

- **Frontend estático optimizado:** Angular genera archivos minificados que reducen ancho de banda y tiempo de carga, disminuyendo el consumo energético del lado del cliente y del servidor.
- **Tecnologías maduras y eficientes:** Java 21 y Spring Boot ofrecen un buen rendimiento por watt; el uso de un servidor embebido reduce la sobrecarga.
- **Servicios cloud con escalado a cero:** se priorizan plataformas que permiten *auto-scaling* y *scale-to-zero* (apagado automático cuando no hay tráfico).

6.3. Monitoreo del rendimiento y consumo de recursos

- Uso previsto de **Spring Boot Actuator** para exponer métricas de salud, memoria y rendimiento.
- Registro de **logs** del backend para detectar cuellos de botella y uso excesivo de recursos.
- En la nube, aprovechar paneles de monitoreo del proveedor (CPU, memoria, red) para ajustar la

capacidad asignada.

6.4. Buenas prácticas en el uso de servicios cloud

- **Autoapagado / scale-to-zero:** apagar instancias sin tráfico para no consumir recursos en vacío.
- **Reutilización:** imágenes de contenedor reutilizables y código modular y mantenible (arquitectura

por capas) que prolonga la vida útil del software y reduce el reproceso.

- **Diseño para eficiencia:** peticiones REST ligeras, paginación de resultados y consultas

optimizadas para minimizar transferencia de datos y carga del servidor.

- **Impacto sostenible:** al ser una tienda virtual, reduce la necesidad de infraestructura física y

de desplazamientos, disminuyendo la huella de carbono frente al comercio tradicional.

7. RESULTADOS Y EVALUACIÓN

7.1. Logros alcanzados frente a los objetivos

Objetivo	Estado	Evidencia
API REST con arquitectura por capas	■ Logrado	6 controladores, servicios y repositorios
Interfaz Angular responsiva	■ Logrado	Catálogo, carrito, login y panel admin
CRUD completo	■ Logrado	CRUD de productos, categorías y banners (UI + API)
Seguridad con JWT y roles	■ Logrado	SecurityConfig, JwtService, @PreAuthorize
Persistencia con JPA/Hibernate	■ Logrado	Entidades mapeadas a SQLite
Documentación de la API	■ Logrado	Swagger UI en /swagger-ui/index.html
Despliegue distribuido documentado	■ Logrado	Sección 5 (Docker, Nginx, SSL, cloud)
Consideraciones de sostenibilidad (ODS 9)	■ Logrado	Sección 6
OAuth2	■ Pendiente	Planteado como mejora futura

7.2. Lecciones aprendidas

- La **separación frontend/backend** facilita el trabajo en equipo en paralelo y el despliegue

independiente, pero exige una buena gestión de **CORS** y de los contratos de la API.

- El uso de **JWT** simplifica la autenticación sin estado, ideal para aplicaciones distribuidas.

- Una **arquitectura por capas** clara reduce el acoplamiento y facilita el mantenimiento.
- Considerar la **eficiencia de recursos** desde el diseño (contenedores ligeros, escalado bajo

demanda) tiene impacto directo en la sostenibilidad y el costo.

- La **documentación temprana** (Swagger, control de versiones) ahorra tiempo y reproceso.

8. CONCLUSIONES Y RECOMENDACIONES

8.1. Conclusiones generales

1. Se desarrolló una aplicación web e-commerce **funcional de extremo a extremo**, cubriendo las fases del ciclo de vida del software: análisis, diseño, desarrollo, pruebas y despliegue.
2. La solución implementa una **arquitectura distribuida** que separa el cliente (Angular) del servidor (Spring Boot), comunicados mediante servicios REST.
3. Se logró un **CRUD completo** y seguro sobre las entidades principales, con control de acceso por roles mediante **Spring Security y JWT**.
4. El diseño desacoplado y el empaquetado en contenedores permiten un **despliegue escalable** en ambientes distribuidos y en la nube.
5. Las decisiones técnicas adoptadas promueven una **infraestructura tecnológica sostenible y eficiente, alineada con el ODS 9**.

8.2. Recomendaciones

- Migrar la base de datos a un motor gestionado (PostgreSQL/MySQL) para producción.
- Implementar **OAuth2** para inicio de sesión con proveedores externos.
- Añadir **Spring Boot Actuator** y un panel de monitoreo de recursos.
- Refactorizar el frontend en componentes y servicios Angular con enrutamiento por URL.
- Automatizar el despliegue con **CI/CD** (GitHub Actions) y contenedores.
- Habilitar **HTTPS** con certificado SSL en el despliegue productivo.

9. BIBLIOGRAFÍA

(Referencias en formato APA 7 — ejemplos; ajustar fechas de consulta)

- Pivotal Software. (2025). *Spring Boot Reference Documentation*. Recuperado de <https://docs.spring.io/spring-boot/>
- VMware. (2025). *Spring Security Reference*. Recuperado de <https://docs.spring.io/spring-security/reference/>
- Google. (2025). *Angular Documentation*. Recuperado de <https://angular.dev/>
- Red Hat. (2025). *Hibernate ORM User Guide*. Recuperado de <https://hibernate.org/orm/documentation/>

- Hipp, D. R. (2025). *SQLite Documentation*. Recuperado de <https://www.sqlite.org/docs.html>
- Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT) — RFC 7519*. IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
- Naciones Unidas. (2015). *Objetivo de Desarrollo Sostenible 9: Industria, Innovación e Infraestructura*. Recuperado de <https://www.un.org/sustainabledevelopment/es/infrastructure/>
- Docker Inc. (2025). *Docker Documentation*. Recuperado de <https://docs.docker.com/>

Documento elaborado como informe final del curso de Soluciones Web y Aplicaciones Distribuidas — Proyecto Bambeli.